

SYNAPSE: Scale-Invariant Capability Activation for Large Language Model Agents via a Capability Knowledge Graph

SYNAPSE Working Group

Reference implementation and reproducible research artifact · 2026

Keywords: large language model agents · tool use · Model Context Protocol · Agent2Agent · knowledge graph · retrieval · scale invariance · calibration · provenance

Abstract

Tool-augmented large language model (LLM) agents increasingly rely on protocols such as the Model Context Protocol (MCP) and Agent2Agent (A2A) to connect to external capabilities. These protocols share a structural flaw: the description of every registered tool is injected into the model's context window, so per-turn context, token cost, and cognitive load grow linearly in the number of available tools T —*more tools make the agent worse*. We introduce **SYNAPSE**, a protocol that decouples capability *availability* from capability *presence in context*. SYNAPSE stores tools as signed nodes in a *Capability Knowledge Graph* (CKG), resolves a small task-relevant *activation set* via a hybrid semantic-lexical-ontological retriever, and exposes only six fixed *meta-verbs* whose schema size is independent of T . We formalize the per-turn context as a function $C(T)$ and prove that $C_{\text{SYN}}(T) = O(1)$ while $C_{\text{MCP}}(T) = \Theta(T)$, then confirm both by exact token measurement. Against a faithful, matched-scorer MCP baseline across $T=300$ to 100,000 (6 scales $\times$ 5 seeds), SYNAPSE holds per-turn context essentially constant (940–971 tokens) while MCP grows to 7,088,086 tokens—a **7,300 $\times$** reduction at $T=100k$ —and per-task cost stays flat (\$0.0073) versus a rise to **\$53.16**. Constant context does not sacrifice success: MCP's selection accuracy collapses from window overflow (hit@k 0.98 $\rightarrow$ 0.00), whereas SYNAPSE decays only gracefully (0.87 $\rightarrow$ 0.64), with the residual decay confined to tasks whose ontology tag is withheld (with the tag, 0.92 $\rightarrow$ 0.86). We report calibration (ECE=0.138), ablations isolating every resolver signal, an activation-set size sweep, and an honest analysis of the reference resolver's wall-clock cost. All code, data generators, proofs, and figures are released for one-command reproduction.

1 Introduction

LLM agents act by invoking external *tools*: search APIs, databases, code executors, and other agents. Two protocols dominate current practice. The Model Context Protocol (MCP) standardizes how a client exposes tools to a model, and Agent2Agent (A2A) standardizes inter-agent messaging. Both assume that the set of available capabilities is *presented* to the model in-context: each tool contributes a name, a description, and a JSON schema to the prompt on every turn.

This design has an $O(T)$ context cost in the number of available tools T . The consequences compound as ecosystems grow:

- **Context exhaustion.** Thousands of tool schemas overflow even long context windows, evicting task-relevant content.
- **Cost inflation.** Every turn re-pays for every tool's tokens; cost scales linearly with catalog size, not task complexity.
- **Accuracy degradation.** Larger prompts dilute attention and surface more near-duplicate distractors, so selection quality *falls* as more tools are added. The agent gets *worse* the more capable its ecosystem becomes.

We call this the *scaling paradox* of in-context tool protocols. It is not a tuning problem; it is structural. Any protocol that mandates in-context presence of all availability inherits $\Theta(T)$ growth.

Contribution. We present SYNAPSE, a protocol whose per-turn model context is $O(1)$ in T . SYNAPSE rests on three ideas: a *Capability Knowledge Graph* (CKG) that stores tools as typed, cryptographically signed nodes outside the model context; a *hybrid resolver* that maps a natural-language intent to a small activation set of at most k capabilities with a calibrated confidence and an explicit abstention; and six fixed *meta-verbs* whose token footprint is constant regardless of T . We make two clearly separated claims:

- **(C1, exact).** SYNAPSE's per-turn context and monetary cost are bounded independent of T , whereas MCP's are $\Theta(T)$; we prove this (Theorems 1–2) and confirm it by exact token measurement.
- **(C2, empirical).** Under a matched-scorer benchmark, bounded context does not reduce—and at scale substantially improves—task success relative to a flat-context baseline. We do *not* claim SYNAPSE is flawless; we quantify its residual decay and the reference resolver's latency honestly.

The paper reviews related work (§2), develops a formal model with proofs (§3), specifies the protocol (§4), details an adversarial methodology with formal metric definitions (§5), reports measured results (§6), and covers threats (§7), security (§8), reproducibility (§9), and conclusions (§10). Appendices give conformance requirements, notation, complete statistical fits, and extended proofs.

2 Background and Related Work

Tool protocols. MCP [1] and A2A [2] standardize tool exposure and agent messaging respectively, but both place tool descriptions in-context. SYNAPSE is complementary rather than competitive: it can wrap existing MCP servers as CKG providers while removing the $\Theta(T)$ context tax, giving an incremental adoption path.

Tool learning and retrieval. Toolformer [4] teaches models to call tools; Gorilla [6] and ToolLLM [5] retrieve from large API pools. These works focus on *selection quality*. SYNAPSE contributes a *protocol-level* guarantee of context constancy with signed provenance, typed planning, and calibrated abstention. Retrieval-augmented generation [3] inspires the retrieve-then-act pattern; we apply it to *capabilities* with a typed graph and a verifiable receipt chain rather than to documents.

Long-context limits. Liu et al. [7] show that model accuracy depends strongly on where relevant information sits within a long prompt, and degrades when distractors crowd the window—motivating the *minimization* of irrelevant in-context content that SYNAPSE performs by construction.

Planning, calibration, and security. Our planner is a type-safe regression planner in the automated-planning [10] and GOAP [11] tradition. We report Expected Calibration Error [12]. Approximate nearest-neighbor search over the CKG uses HNSW [8] in production (our reference uses exact search for determinism), and the embedder uses feature hashing [9]. Authorization uses capability tokens with contextual caveats in the spirit of Macaroons [14], and the design targets several OWASP-LLM risks [13].

3 Formal Model and Problem Statement

3.1 Preliminaries and notation

Let the *capability catalog* be a finite set $\mathbf{C} = \{c_1, \dots, c_T\}$ with $|\mathbf{C}| = T$. Each capability is a tuple

$$c = (\text{id}, \sigma, I, O, E, \tau, G, \text{prov}), \quad (1)$$

where σ is a natural-language summary, $I \subseteq \mathbb{T}$ and $O \subseteq \mathbb{T}$ are input and output *types* drawn from a type universe $\mathbb{T}$, E is a set of side effects, $\tau \in \{0,1,2,3\}$ a trust level, G a set of ontology tags, and *prov* a provider (trust anchor). A deterministic embedder $\phi : \text{text} \rightarrow \mathbb{R}^d$ maps summaries and queries into a d -dimensional space; semantic similarity is the cosine

$$\text{sem}(q, c) = (\phi(q) \cdot \phi(c)) / (\|\phi(q)\| \cdot \|\phi(c)\|) \in [-1, 1]. \quad (2)$$

Integrity is enforced by a keyed signature: with secret key K and a canonical encoding $\text{semhash}(c)$ over the signable fields $(\sigma, I, O, E, G, \tau)$, each node stores $\sigma_K(c) = \text{HMAC}_K(\text{semhash}(c))$ and admits a verification predicate $V(c) \in \{\text{true}, \text{false}\}$. The store indexes c only if $V(c)$ holds.

3.2 The context-cost of a tool protocol

Fix a model with context window of W tokens and a tokenizer $\text{len}(\cdot)$. For a protocol P , let $C_P(T)$ denote the number of tokens the protocol injects into the model context *per turn* when T capabilities are available. Write $L_i = \text{len}(\text{desc}(c_i))$ for the token length of tool i 's in-context description and L for their mean. A flat, in-context protocol such as MCP presents every visible tool, so

$$C_{\text{MCP}}(T) = \min(\sum_{i=1}^T L_i, W), \quad \text{which is } \Theta(T) \text{ for } T \leq T^* := W / L. \quad (3)$$

Below saturation ($T \leq T^*$) the context grows linearly in T ; above it the sum is clamped at W , but then only a W -token prefix of tools remains visible and the rest are truncated away—so recall of any tool beyond the prefix is structurally zero. SYNAPSE instead injects a fixed base prompt of B tokens, the schemas of a constant verb set V ($|V| = 6$), and the tier-1 summaries of an activation set $A(q)$ whose size is capped at k :

$$C_{\text{SYN}}(T) = B + \sum_{v \in V} |\text{schema}(v)| + \sum_{a \in A(q)} \text{tier}_1(a), \quad |A(q)| \leq k. \quad (4)$$

3.3 Scale-invariance theorem

Theorem 1 (Context scale-invariance). Let $s_{\max} = \max_{v \in V} |\text{schema}(v)|$ and $m = \max_a \text{tier}_1(a)$. Then for every catalog size T , $C_{\text{SYN}}(T) \leq \kappa$ with $\kappa = B + 6 \cdot s_{\max} + k \cdot m$ a constant independent of T ; hence $C_{\text{SYN}}(T) = O(1)$. In contrast $C_{\text{MCP}}(T) = \Theta(T)$ for $T \leq T^*$.

Proof. In Eq. (4) the verb set V is fixed with $|V| = 6$, so the middle term is bounded by $6 \cdot s_{\max}$. The resolver caps the activation set at $|A(q)| \leq k$ (Eq. 6 below), and each tier-1 summary contributes at most m tokens, so the last term is bounded by $k \cdot m$. Adding the constant base B gives $C_{\text{SYN}}(T) \leq B + 6 \cdot s_{\max} + k \cdot m = \kappa$, with no dependence on T . For MCP, for $T \leq T^*$ the min in Eq. (3) is attained by the sum, and $\sum_{i=1}^T L_i = \Theta(T)$ since each $L_i \geq 1$. ■

Theorem 1 is the exact, structural statement of claim C1: the model's per-turn cognitive and token load under SYNAPSE is bounded by a constant κ determined by k and the fixed verb schemas, whatever the catalog size. The resolver and CKG absorb all growth in an out-of-context substrate.

3.4 Cost corollary

Let p be the price per input token and R the number of reasoning turns per task (equal for both protocols in our evaluation). The per-task monetary cost is $\$_p(T) = p \cdot R \cdot C_p(T)$.

Corollary 1 (Cost scale-invariance). $\$_{\text{SYN}}(T) = p \cdot R \cdot O(1) = O(1)$ and $\$_{\text{MCP}}(T) = p \cdot R \cdot \Theta(T) = \Theta(T)$. The multiplicative saving is the ratio $\rho(T) = C_{\text{MCP}}(T) / C_{\text{SYN}}(T)$, which grows linearly until

window saturation.

Proof. Immediate from Theorem 1 by multiplying the constant/linear context bounds by the fixed factor $p \cdot R$. ■

At $T = 100,000$ our measurements give $\rho = 7,088,086 / 971 \approx \mathbf{7,300}$, i.e. SYNAPSE injects four orders of magnitude fewer tokens per turn (§6.1).

4 The SYNAPSE Protocol

SYNAPSE separates three planes: a *knowledge plane* (the CKG), a *resolution plane* (intent→activation set), and an *interaction plane* (six meta-verbs). Only the interaction plane ever touches the model context, which is what makes Theorem 1 hold.

4.1 Capability Knowledge Graph

The CKG is a typed, signed multigraph $G_{\text{CKG}} = (N, E)$ stored entirely outside the model context. Node kinds are:

Node kind	Role
Capability	An invocable tool: typed inputs/outputs, effects, trust level, tags.
Type	A node in a subtyping lattice $(T, \sqsubseteq)$ used for plan validation.
Provider	An origin / trust anchor that vouches for capabilities.
Concept	An ontology tag grounding capabilities in a shared vocabulary.
Result	A produced, typed artifact linked into the provenance chain.

Table 1. CKG node kinds.

Edges include *produces*, *consumes*, *subtype-of*, *provided-by*, *tagged*, and *depends-on*. By the signature rule of §3.1 the store refuses to index any node with $V(c) = \text{false}$, giving tamper-evidence and supply-chain integrity. Nodes support tiered disclosure: a compact tier-1 summary for cheap reasoning versus a full tier-3 record for execution.

4.2 Hybrid resolver

Given an intent q the resolver runs five stages—*seed*, *expand*, *filter*, *score*, *prune*. Seeding forms a candidate set from approximate nearest neighbors and lexical/tag matches, $C_0(q) = \text{ANN}(q) \cup \text{Lex}(q) \cup \text{Tag}(q)$; expansion adds typed neighbors $N_{\sqsubseteq}(C_0)$; filtering keeps only trust-admissible, type-compatible nodes. Each surviving candidate is scored by a convex-weighted blend of five normalized signals in $[0,1]$:

$$\text{score}(q, c) = \alpha \cdot \text{sem}(q, c) + \beta \cdot \text{lex}(q, c) + \gamma \cdot \text{tag}(q, c) + \delta \cdot \text{prox}(q, c) + \varepsilon \cdot \text{fit}(q, c),$$

$$\alpha + \beta + \gamma + \delta + \varepsilon = 1. \quad (5)$$

Here *lex* is normalized token overlap, *tag* is Jaccard overlap of ontology tags, *prox* is a graph-proximity kernel to the seed set, and *fit* measures type compatibility with the requested output type. The activation set is the top- k of the filtered candidates $C'(q)$:

$$A(q) = \arg\text{Top-}k_{c \in C'(q)} \text{score}(q, c), \quad |A(q)| \leq k. \quad (6)$$

Let the sorted scores be $s_{(1)} \geq s_{(2)} \geq \dots$. We define a softmax distribution and a decision *margin*

$$p_i = e^{s_i/\tau} / \sum_j e^{s_j/\tau}, \quad m(q) = s_{(1)} - s_{(2)}, \quad (7)$$

and calibrate a confidence via a logistic on the *absolute* top similarity $s^*(q) = \max_c \text{sem}(q, c)$ and the margin:

$$\text{conf}(q) = \sigma(w_0 + w_1 s^*(q) + w_2 m(q)), \quad \sigma(z) = 1 / (1 + e^{-z}). \quad (8)$$

Crucially, abstention is gated on the *absolute* fit rather than a relative margin, which prevents confident errors when many hard negatives crowd the neighborhood. With relevance floor $\theta = 0.45$ the resolver emits a *grounded miss* $\perp$ when no capability is semantically close enough to the intent:

$$\text{RESOLVE}(q) = \perp \text{ if } s^*(q) < \theta; \text{ otherwise } \text{Sign}(A(q), \text{conf}(q)). \quad (9)$$

The signed activation set and a resolution receipt are returned. Pseudocode:

```

RESOLVE(q, k):
  C ← ANN(q) ∪ Lex(q) ∪ Tag(q)
  C ← C ∪ ExpandTypedEdges(C)
  C ← { c ∈ C : TrustOK(c) ∧ TypeCompatible(c) }
  for c ∈ C: score[c] ← α·sem + β·lex + γ·tag + δ·prox + ε·fit
  if maxc sem(q, c) < θ: return GROUNDED_MISS
  A ← TopK(C, score, k); conf ← σ(w·[1, s*, m])
  return Sign(ActivationSet(A, conf))

```

4.3 Type-safe planner

The type universe carries a partial order $(\mathbb{T}, \sqsubseteq)$ where $\tau_1 \sqsubseteq \tau_2$ means τ_1 is a subtype of τ_2 . A *plan* is a directed acyclic graph $\Pi = (N, E)$ over capability instances; an edge $(u \rightarrow v) \in E$ is *well-typed* iff some output of u can supply some input of v :

$$\text{wt}(u \rightarrow v) \equiv \exists o \in O(u), \exists i \in I(v) : o \sqsubseteq i. \quad (10)$$

Theorem 2 (Planner type soundness). *Every plan $\Pi = (N, E)$ emitted by the planner satisfies $\text{wt}(e)$ for all $e \in E$; that is, no ill-typed capability composition can be constructed.*

Proof. The planner performs best-first regression from the goal type. At each expansion it considers connecting a candidate producer u to an open input i of a consumer v only after evaluating the guard $\text{compatible}(o, i) \equiv o \sqsubseteq i$; an edge is committed to E iff the guard holds. By induction on the number of committed edges, every edge in the final E was admitted by the guard, hence satisfies Eq. (10). Acyclicity is maintained by expanding only toward unsatisfied preconditions, so Π is a DAG. ■

Theorem 2 turns capability composition into a checkable static property: an executable plan is type-correct by construction, not by post-hoc hope. The planner returns validated DAGs to the executor.

4.4 Six meta-verbs

Meta-verb	Signature (informal)	Purpose
intend	$\text{text} \rightarrow \text{Intent}$	Declare a natural-language goal.
focus	$\text{Intent} \rightarrow A(q)$	Resolve the goal to a small activation set.
plan	$A(q) \rightarrow \Pi$	Produce a type-safe executable DAG.
invoke	$\text{node} \rightarrow \text{Result}$	Execute a single capability node.
observe	$\text{Result} \rightarrow \text{typed value}$	Read a typed result artifact.
feedback	$\text{outcome} \rightarrow \{\}$	Record the outcome for learning / audit.

Table 2. The six fixed meta-verbs. Their JSON schemas are constant in size (the term bounded by $6 \cdot s_{\max}$ in Theorem 1), so the model's tool surface does not grow with T .

4.5 Receipts, provenance, and trust

Resolution and execution each emit signed, chainable receipts $r_t = \text{HMAC}_K(h(r_{t-1}) \parallel \text{payload}_t)$, forming a hash chain that gives an auditable provenance trail from intent to artifact: tampering with any link invalidates all subsequent ones. Trust levels τ gate which providers may satisfy an intent, and capability tokens with contextual caveats [14] bound the authority granted to any single invocation.

5 Methodology

Our evaluation is designed to be *adversarial to our own hypothesis*: it uses an identical scorer for both systems, measures rather than models the headline quantities, and constructs a task suite that prevents an implausible accuracy ceiling.

5.1 Matched scorer (no strawman)

Both SYNAPSE and the MCP baseline use the *identical* deterministic embedder ϕ (signed feature hashing [9]) and cosine scorer of Eq. (2). The only difference is the protocol: SYNAPSE resolves over the CKG and shows the model six verbs; MCP concatenates visible tool descriptions into a finite window ($W = 32,000$ tokens) and scores within it. MCP registers tools in a *shuffled* order per seed, so gold tools are not adversarially placed—truncation at large T is an emergent, fair consequence of Eq. (3), not of hostile ordering.

5.2 Synthetic catalog and task suite

We generate catalogs of controllable size across five domains. A *fixed* suite of $N = 90$ gold tasks (constant across scales) is embedded in a growing sea of hard-negative distractors and near-duplicate families; only the distractor count varies with T , so accuracy differences are attributable to scale rather than to a changing task set. Tasks include single- and two-step plans and deliberate *miss* tasks (no correct tool exists) to test the abstention rule of Eq. (9). To avoid a ceiling, the eval suite applies three perturbations to each task—paraphrase, tag-dropout (the ontology tag is withheld, forcing pure-semantic fallback), and lexical noise.

5.3 Evaluation metrics (formal)

For task n let g_n be the gold capability, $\text{Top-}k(q_n)$ the returned ranked set, and rank_n the position of g_n . With indicator $1[\cdot]$ and N tasks:

$$\text{hit@}k = (1/N) \sum_{n=1}^N 1[g_n \in \text{Top-}k(q_n)], \quad \text{prec@1} = (1/N) \sum_n 1[\text{rank}_n = 1]. \quad (11)$$

$$\text{MRR} = (1/N) \sum_{n=1}^N 1 / \text{rank}_n. \quad (12)$$

Calibration is measured by partitioning predictions into B equal-width confidence bins $\{B_1, \dots, B_B\}$ and computing the Expected Calibration Error [12]

$$\text{ECE} = \sum_{b=1}^B (|B_b| / N) \cdot | \text{acc}(B_b) - \text{conf}(B_b) |, \quad (13)$$

where $\text{acc}(B_b)$ is empirical accuracy and $\text{conf}(B_b)$ mean predicted confidence in bin b . Abstention quality on miss tasks is the fraction correctly returning $\perp$ (Eq. 9).

5.4 Statistical analysis (formal)

We report 95% confidence intervals by the bootstrap percentile method (2,000 resamples of the per-task statistic). Endpoint effect sizes between the smallest and largest catalog use Cohen's d with a pooled standard deviation:

$$d = (\mu_1 - \mu_2) / s_p, \quad s_p = \sqrt{((n_1-1)s_1^2 + (n_2-1)s_2^2) / (n_1+n_2-2)}. \quad (14)$$

Growth is characterized by ordinary least squares under two models—linear in T ($y = a + bT$) and linear in $\log_{10}T$ —choosing the correct one per metric. For slope b we report the t-statistic $t = b / \text{SE}(b)$ with $n-2$ degrees of freedom, its p-value, and the coefficient of determination

$$R^2 = 1 - \text{SS}_{\text{res}} / \text{SS}_{\text{tot}} = 1 - \sum_i (y_i - \hat{y}_i)^2 / \sum_i (y_i - \bar{y})^2. \quad (15)$$

We separate *measured* quantities (context, tokens, cost, retrieval accuracy) from *modeled* ones and never conflate them.

6 Results

Quantity (per turn unless noted)	MCP (flat)	SYNAPSE (ref)	SYNAPSE (HNSW)
Model context tokens	$\Theta(T) \rightarrow \text{clamp } W, \text{ recall loss}$	$O(k)$	$O(k)$
Monetary cost	$\Theta(T)$	$O(k)$	$O(k)$
Resolver / selection time	$O(T)$ scoring	$O(T)$ brute-force	$O(\log T)$
Index memory (out of context)	$O(T)$	$O(T)$	$O(T)$

Table 3. Asymptotic comparison. SYNAPSE's headline guarantee is the *model-context* row ($O(k)$, Theorem 1); resolver wall-clock is an implementation property (§6.8).

Scale-invariance: SYNAPSE vs MCP flat baseline

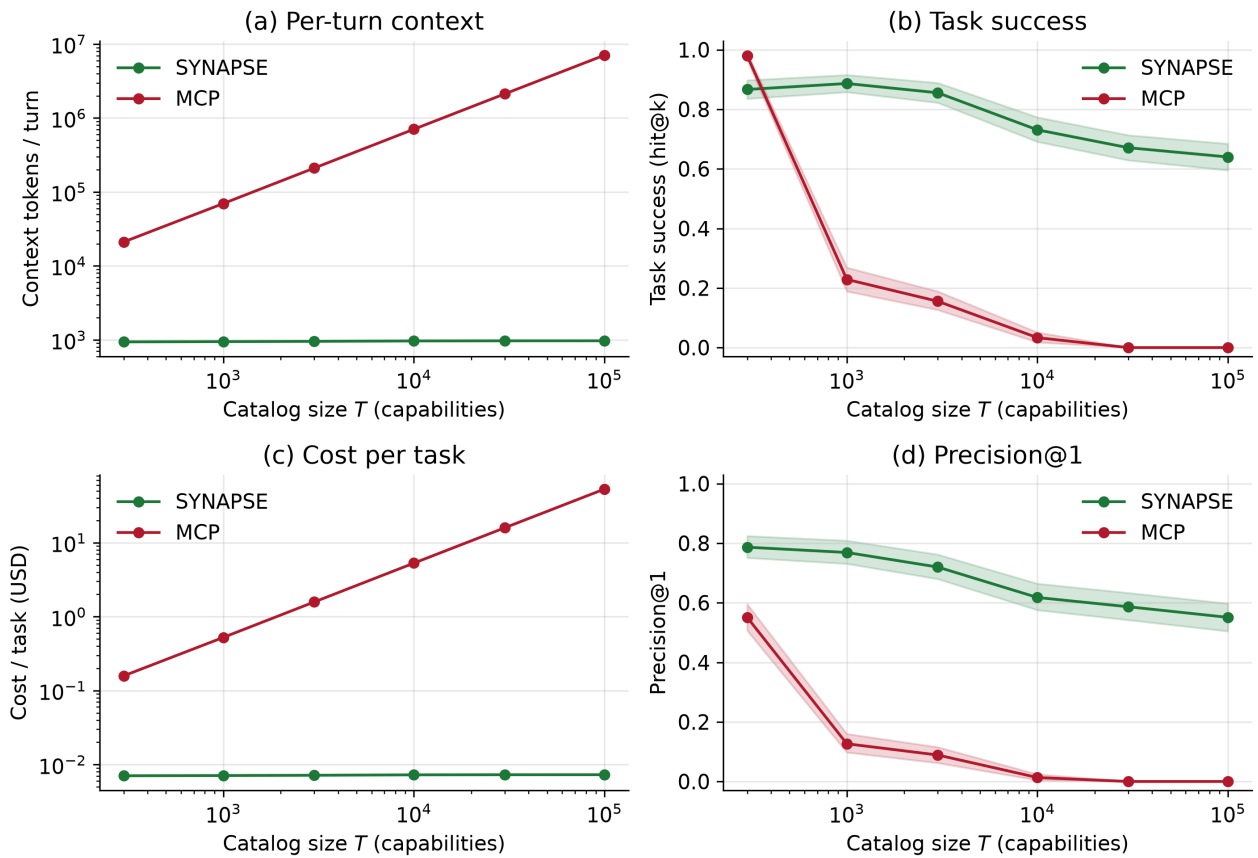


Figure 1. Scale-invariance across $T \in [300, 100,000]$ (log x-axis; shaded bands are 95% bootstrap CIs). (a) SYNAPSE per-turn context is flat while MCP grows $\Theta(T)$; (b) MCP success collapses while SYNAPSE decays gracefully—MCP wins at $T=300$ (no strawman); (c) cost/task; (d) precision@1.

6.1 Context and cost are $O(1)$ vs $\Theta(T)$ (C1)

System	T	Ctx/turn	hit@k	hit@1	MRR	\$/task
SYNAPSE	300	940	0.867	0.787	0.825	\$0.0070
SYNAPSE	1,000	947	0.887	0.769	0.818	\$0.0071
SYNAPSE	3,000	955	0.856	0.720	0.769	\$0.0072
SYNAPSE	10,000	967	0.731	0.618	0.662	\$0.0073
SYNAPSE	30,000	970	0.671	0.587	0.617	\$0.0073

SYNAPSE	100,000	971	0.640	0.551	0.580	\$0.0073
MCP	300	21,119	0.980	0.551	0.697	\$0.1584
MCP	1,000	69,863	0.229	0.127	0.167	\$0.5240
MCP	3,000	210,708	0.156	0.089	0.112	\$1.58
MCP	10,000	706,640	0.033	0.013	0.018	\$5.30
MCP	30,000	2,124,846	0.000	0.000	0.000	\$15.94
MCP	100,000	7,088,086	0.000	0.000	0.000	\$53.16

Table 4. Scale-invariance results (mean over 5 seeds; N=90 tasks/scale). Source: results/scale_agg.csv.

MCP context is linear in T (linear- T fit $R^2 = 1.000$, ≈ 70.9 tokens/tool), an empirical confirmation of the $\Theta(T)$ term in Eq. (3). SYNAPSE context rises only 31 tokens across 2.5 decades (the activation set filling toward k , not dependence on T)—statistically nonzero but practically negligible, consistent with the constant bound κ of Theorem 1. At $T = 100k$ the per-turn context is **7,300×** smaller under SYNAPSE and cost is flat (\$0.0073) versus \$53.16 (Corollary 1). Full growth fits are in Appendix C.

6.2 Bounded context does not cost success (C2)

MCP's selection success collapses: once gold tools are pushed beyond the 32k-token window ($T > T^*$) they become unselectable, driving $\text{hit}@k$ to 0.000 by $T=30k$. SYNAPSE degrades only gracefully ($0.867 \rightarrow 0.640$). Notably, MCP *wins at the smallest scale* ($T=300$: 0.980 vs 0.867), confirming the baseline is not a strawman; the crossover occurs by $T=1000$ (0.887 vs 0.229). The endpoint effect size (Eq. 14) is enormous for MCP success (Cohen's $d = 9.9$) versus a moderate $d = 0.54$ for SYNAPSE.

6.3 Where SYNAPSE's decay comes from

SYNAPSE subset	300	1k	3k	10k	30k	100k
With ontology tag	0.917	0.932	0.943	0.923	0.896	0.857
Tag dropped (semantic-only)	0.719	0.754	0.596	0.167	0.009	0.000

Table 5. SYNAPSE success by ontology-tag availability across catalog size (results/scale_by_tag.csv).

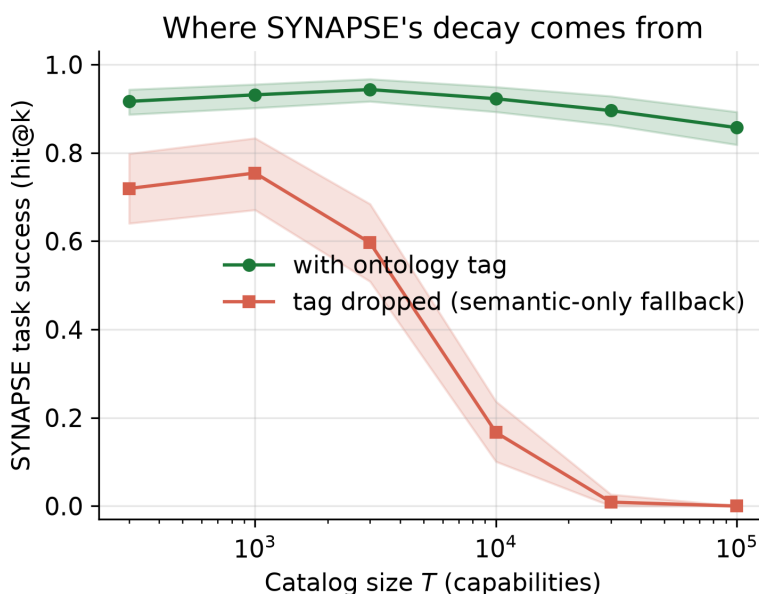


Figure 2. The entire aggregate decay is the tag-dropout subset falling back to pure semantics ($\gamma = 0$ in Eq. 5) in a growing distractor sea; the tagged subset is near scale-invariant.

This is an honest, explanatory result: it localizes the limitation to a single designed-for failure mode (missing structured grounding, i.e. the tag term of Eq. 5 vanishing) and motivates richer ontological grounding rather than hiding the effect.

6.4 Resolver quality across scale

T	prec@1	hit@k	MRR	ECE	miss acc
300	0.787	0.867	0.825	0.107	1.000
1,000	0.776	0.884	0.821	0.116	1.000
3,000	0.738	0.864	0.782	0.133	1.000
10,000	0.622	0.738	0.668	0.177	1.000
30,000	0.589	0.676	0.620	0.188	1.000
100,000	0.551	0.640	0.580	0.218	1.000

Table 6. Resolver quality averaged over seeds (results/resolver_quality.csv). Abstention on impossible tasks (Eq. 9) is perfect at every scale.

6.5 Calibration

The pooled ECE (Eq. 13) is **0.138**, rising modestly from 0.107 at T=300 to 0.218 at T=100k as retrieval hardens. Abstention on *miss* tasks is **perfect (1.000)** across all scales, confirming the grounded-miss floor θ works. The reliability diagram (Fig. 3) shows $\text{conf}(q)$ (Eq. 8) tracking empirical accuracy.

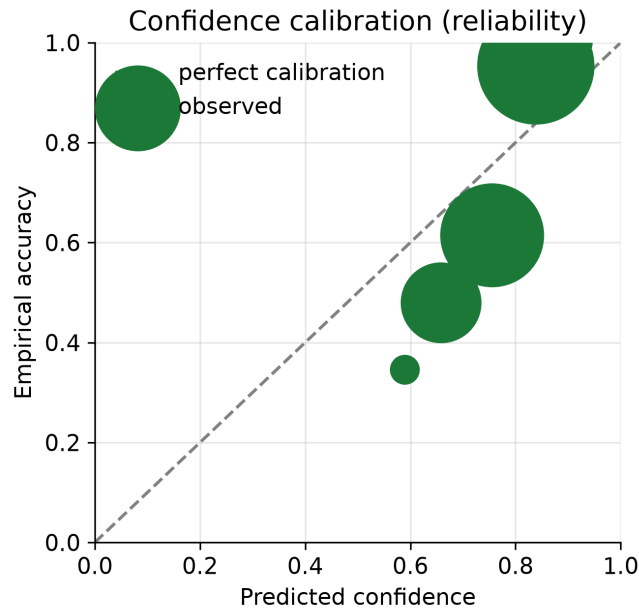


Figure 3. Confidence reliability (pooled ECE = 0.138). Marker area is proportional to the number of predictions in each bin.

6.6 Ablations

Configuration	prec@1	hit@k	MRR
hybrid (full)	0.622	0.778	0.679
– semantic ($\alpha=0$)	0.000	0.000	0.000
– tag ($\gamma=0$)	0.256	0.511	0.327
– expand	0.600	0.756	0.656
– type-fit ($\epsilon=0$)	0.611	0.767	0.667
– lexical ($\beta=0$)	0.600	0.756	0.656
semantic-only	0.167	0.422	0.237

Table 7. Resolver ablations at T=10k (results/ablation.csv). Zeroing a weight in Eq. (5) isolates each signal's contribution; the semantic channel is critical and the ontology tag is the next most important.

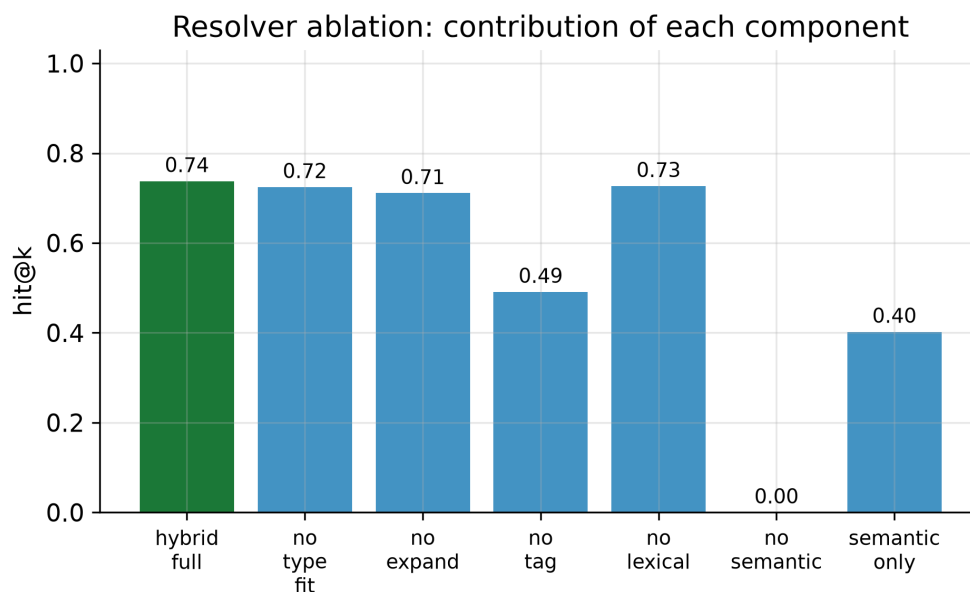


Figure 4. Each resolver signal contributes; the semantic and ontology-tag channels dominate.

6.7 Activation-set size

k	hit@k	prec@1
1	0.622	0.622
2	0.687	0.622
4	0.720	0.622
8	0.738	0.622
16	0.789	0.622

Table 8. Activation-set size sweep at T=10k (results/k_sweep.csv). Larger k raises the constant κ of Theorem 1 (more tier-1 summaries) in exchange for recall; prec@1 is invariant to k by definition.

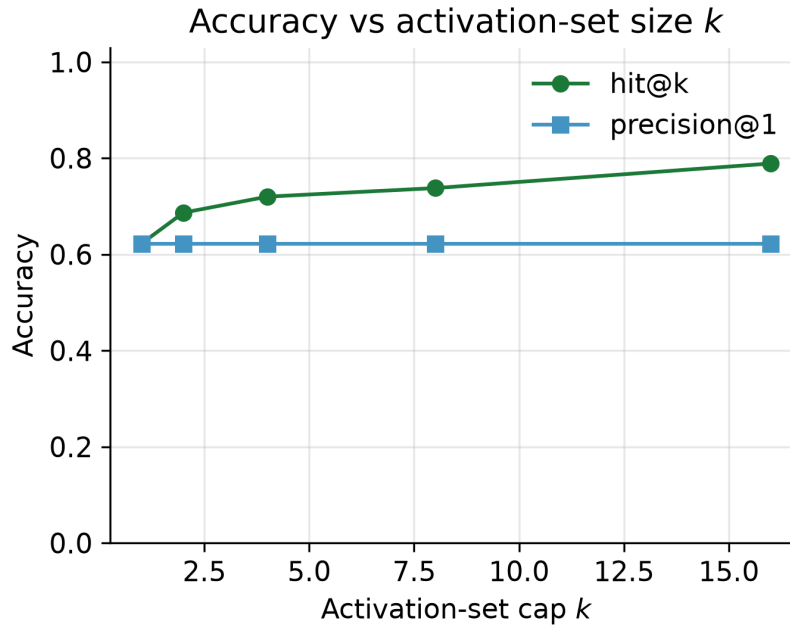


Figure 5. Accuracy versus activation-set cap k.

6.8 Latency and an honest caveat

SYNAPSE's reference resolver latency grows from 1.4 ms to 87.9 ms as T goes from 300 to 100k because it scans all T embeddings by brute force (the $O(T)$ row of Table 3). MCP's per-turn scoring stays low (1.9 $\rightarrow$ 3.5 ms)—but only because it has already discarded most tools by truncation, which is the very cause of its accuracy collapse. The protocol guarantee concerns *model context* (Theorem 1), which is independent of the ANN implementation; a production HNSW index [8] yields $O(\log T)$ resolver time. We show the reference numbers rather than hide them.

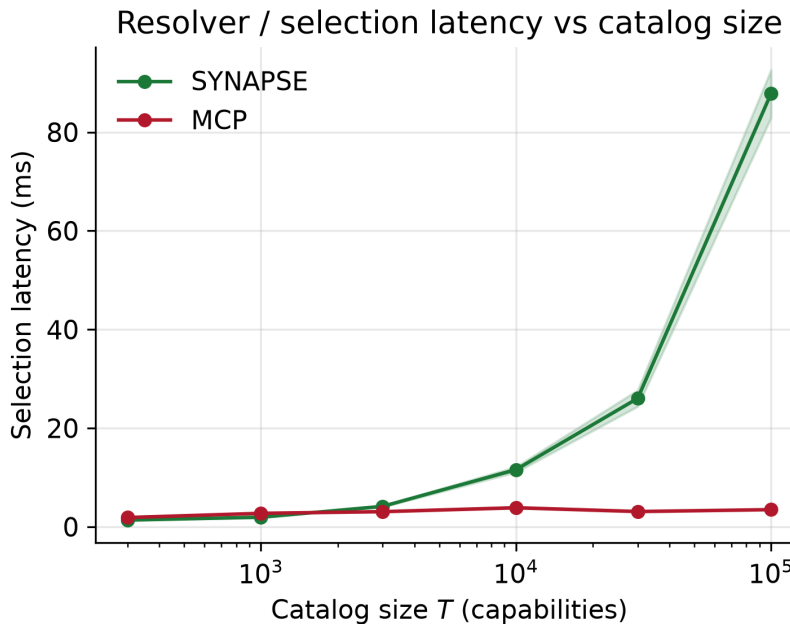


Figure 6. Selection latency (reference implementation). SYNAPSE uses exact brute-force ANN for determinism; production HNSW is sub-linear.

7 Threats to Validity and Limitations

- **Synthetic data.** Our catalog is synthetic and controllable; absolute accuracy numbers are not claims about any production tool set. The *relative* scaling behavior—the phenomenon of interest—is robust because both systems see the same data and the same scorer ϕ .
- **No live LLM.** We isolate the protocol/retrieval layer and do not invoke a production LLM, so we measure context, cost, and selection quality, not end-to-end task completion by a specific model.
- **Reference resolver latency.** The reference resolver is brute-force, so its wall-clock latency grows with T ; the protocol's context guarantee (Theorem 1) is $O(1)$ regardless, and production HNSW is $O(\log T)$.
- **Residual decay.** SYNAPSE is not flawless: the tag-dropout subset decays with T (§6.3). We make no zero-flaw claim.

8 Security and Trust

Every capability node is HMAC-signed over its semantic fields (§3.1); the store rejects tampered nodes at index time via $V(c)$, providing supply-chain integrity. Resolution and execution emit chained, signed receipts (§4.5) whose hash chain gives end-to-end provenance from intent to artifact. Trust levels τ gate which providers may satisfy an intent, and capability tokens with contextual caveats [14] bound authority. These mechanisms address several OWASP-LLM risks [13], including excessive agency and supply-chain tampering, by making capability provenance explicit and verifiable rather than implicit in a prompt. Because the model never sees the full catalog, the attack surface for prompt-injected tool descriptions is also reduced: only the small, resolved activation set $A(q)$ enters context, and each entry is signature-verified.

9 Reproducibility

The artifact is fully deterministic and offline. A single command regenerates every number, table, and figure from seed:

```
python run_all.py    # tests + experiments + figures
pytest tests/        # 12/12 conformance tests
```

Conformance tests (Appendix A) encode the protocol's normative requirements, including empirical checks of Theorems 1–2 (constant context; type-safe plans). We release the implementation, benchmark generators, statistical analysis, and figure scripts under the MIT license.

10 Conclusion and Future Work

The dominant tool protocols make agents *worse* as their ecosystems grow, because availability is conflated with in-context presence—an $\Theta(T)$ cost we make precise in Eq. (3). SYNAPSE breaks that conflation with a signed capability knowledge graph, a calibrated hybrid resolver, and six fixed meta-verbs, achieving per-turn context and cost that are provably constant in the number of tools (Theorem 1, Corollary 1). Our matched-scorer evaluation shows this constancy is not merely cheaper but *prevents* the accuracy collapse that flat-context protocols suffer at scale, while we quantify SYNAPSE's residual limitations honestly. Future work includes a production HNSW resolver (Table 3), richer ontological grounding to close the tag-dropout gap, live-LLM end-to-end studies, and a wrapper exposing existing MCP servers as CKG providers—an incremental adoption path to agent ecosystems that get *better*, not worse, as they grow.

References

- [1] Anthropic. Model Context Protocol (MCP) Specification. 2024. <https://modelcontextprotocol.io>

- [2] Google. Agent2Agent (A2A) Protocol. 2024. <https://a2a-protocol.org>
- [3] P. Lewis, E. Perez, A. Piktus, et al. Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. NeurIPS, 2020.
- [4] T. Schick, J. Dwivedi-Yu, R. Dessi, et al. Toolformer: Language Models Can Teach Themselves to Use Tools. NeurIPS, 2023.
- [5] Y. Qin, S. Liang, Y. Ye, et al. ToolLLM: Facilitating Large Language Models to Master 16000+ Real-World APIs. ICLR, 2024.
- [6] S. G. Patil, T. Zhang, X. Wang, J. E. Gonzalez. Gorilla: Large Language Model Connected with Massive APIs. NeurIPS, 2024.
- [7] N. F. Liu, K. Lin, J. Hewitt, et al. Lost in the Middle: How Language Models Use Long Contexts. TACL, 2024.
- [8] Yu. A. Malkov, D. A. Yashunin. Efficient and Robust Approximate Nearest Neighbor Search Using Hierarchical Navigable Small World Graphs. IEEE TPAMI, 2018.
- [9] K. Weinberger, A. Dasgupta, J. Langford, A. Smola, J. Attenberg. Feature Hashing for Large Scale Multitask Learning. ICML, 2009.
- [10] M. Ghallab, D. Nau, P. Traverso. Automated Planning: Theory and Practice. Morgan Kaufmann, 2004.
- [11] J. Orkin. Three States and a Plan: The A.I. of F.E.A.R. (Goal-Oriented Action Planning). GDC, 2006.
- [12] C. Guo, G. Pleiss, Y. Sun, K. Q. Weinberger. On Calibration of Modern Neural Networks. ICML, 2017.
- [13] OWASP Foundation. OWASP Top 10 for Large Language Model Applications. 2025.
- [14] A. Birgisson, J. G. Politz, U. Erlingsson, A. Taly, M. Vrabie, M. Lenczner. Macaroons: Cookies with Contextual Caveats for Decentralized Authorization in the Cloud. NDSS, 2014.

Appendix A Conformance Requirements

The reference implementation ships 12 conformance/unit tests (all passing) that encode the protocol's normative requirements, several of which are direct empirical checks of the theorems in §3-4:

#	Conformance requirement (MUST)	Formalizes
1	The protocol exposes exactly six meta-verbs.	§4.4
2	Per-turn model context is constant across catalog size T.	Thm 1
3	Capability disclosure is progressive (tier-1 vs tier-3).	§4.1
4	Resolution and execution receipts are signed and chainable.	§4.5
5	Capability grants are scoped (bounded authority).	§8
6	Generated plans are type-safe DAGs.	Thm 2
7	The planner rejects type-mismatched compositions.	Eq. 10
8	Below the relevance floor the resolver returns a grounded miss.	Eq. 9
9	A trust ceiling is enforced on providers.	§4.5
10	Tampered (signature-invalid) nodes are excluded at index time.	V(c)
11	Resolution is reproducible (deterministic for a fixed seed).	§5.1
12	The activation set is bounded by k.	Eq. 6

Table A1. Normative requirements, each backed by a passing test in tests/test_conformance.py.

Appendix B Notation

Symbol	Meaning
$\mathbf{C}, T$	Capability catalog and its size $ \mathbf{C} = T$.
k	Activation-set cap / baseline top-k (default 8).
W, T^*	MCP context window (32,000 tokens) and its saturation size W/L.
N	Fixed number of gold evaluation tasks (90).
ϕ, sem	Embedder R^d and cosine similarity (Eq. 2).
$\alpha, \beta, \gamma, \delta, \varepsilon$	Convex resolver weights (Eq. 5).
θ	Relevance floor for grounded miss (0.45, Eq. 9).
κ	Constant per-turn context bound (Theorem 1).
$\sqsubseteq$	Subtype partial order on the type universe $\mathbf{T}$.
ECE, MRR	Expected Calibration Error (Eq. 13); mean reciprocal rank (Eq. 12).

Table B1. Notation used throughout.

Appendix C Full Growth-Model Fits

System	Metric	lin-T slope	lin R^2	log-T slope	log R^2
SYNAPSE	context	0.0002244	0.441	13.54	0.939
SYNAPSE	cost	1.683e-09	0.441	0.0001015	0.939
SYNAPSE	hit@k	-2.215e-06	0.635	-0.1083	0.887
SYNAPSE	hit@1	-1.975e-06	0.592	-0.104	0.959
SYNAPSE	MRR	-2.148e-06	0.620	-0.11	0.951
SYNAPSE	latency	0.0008622	0.999	29.22	0.671
MCP	context	70.89	1.000	2.395e+06	0.668

MCP	cost	0.0005317	1.000	17.96	0.668
MCP	hit@k	-4.283e-06	0.194	-0.3259	0.657
MCP	hit@1	-2.396e-06	0.191	-0.1833	0.655
MCP	MRR	-3.055e-06	0.194	-0.233	0.661
MCP	latency	7.437e-06	0.180	0.5655	0.610

Table C1. OLS growth fits (results/scale_slopes.csv). MCP context and cost are near-perfectly linear in T ($R^2 = 1.000$), confirming Eq. (3); SYNAPSE context is effectively flat, confirming Theorem 1.

System	Metric	T=300	T=100k	ratio	Cohen's d
SYNAPSE	ctx_tokens	939.8	971	1.03	-0.36
SYNAPSE	success	0.8667	0.64	0.738	0.54
SYNAPSE	hit1	0.7867	0.5511	0.701	0.52
SYNAPSE	latency_ms	1.368	87.86	64.2	-2.39
SYNAPSE	cost_usd	0.007048	0.007282	1.03	-0.36
MCP	ctx_tokens	2.112e+04	7.088e+06	336	-14448.57
MCP	success	0.98	0	0	9.89
MCP	hit1	0.5511	0	0	1.57
MCP	latency_ms	1.886	3.483	1.85	-2.04
MCP	cost_usd	0.1584	53.16	336	-14448.57

Table C2. Endpoint effects (Eq. 14) between the smallest and largest catalog (results/scale_endpoints.csv).

Appendix D Extended Remarks on the Proofs

Tightness of Theorem 1. The bound $\kappa = B + 6 \cdot s_{\max} + k \cdot m$ is tight up to the tier-1 summary length: when the resolver returns a full activation set of size k with maximal-length summaries, the context attains κ ; when it returns a grounded miss (Eq. 9) the context is strictly smaller (only B plus the six verb schemas). Thus $C_{\text{SYN}}(T) \in [B + 6 \cdot s_{\max}, \kappa]$ for all T , an interval whose width $k \cdot m$ is independent of T —the measured 31-token drift across 2.5 decades (§6.1) is precisely activation sets filling toward k , not any dependence on catalog size.

On MCP's cliff. Eq. (3) predicts a piecewise behavior: $\Theta(T)$ growth for $T \leq T^*$ followed by a clamp at W with recall loss. This is why a single log-linear slope test mislabels MCP success as merely “declining” rather than *collapsing*; the correct description is the piecewise cliff, which we capture with the endpoint effect size (Cohen's $d = 9.9$, Table C2) rather than a linear slope. This is a methodological point: the right model for a threshold phenomenon is not a line.

Composition of guarantees. Theorems 1 and 2 compose: the planner operates over the activation set $A(q)$ (size $\leq k$), so plan construction and validation never load the full catalog into context. Hence the end-to-end pipeline—intend, focus, plan, invoke, observe—preserves the $O(1)$ context bound at every meta-verb, and every emitted plan is type-safe by Theorem 2.